UAX	ESTRUCTURA DE COMPUTADORES
Nombre:_____	

Problema 1 (2,5 puntos)

Se ejecutó un programa de prueba en un procesador RISC-V de 32 bits (RV32I) de ciclo múltiple, implementado en una FPGA y funcionando a 150 MHz. La tabla recoge la distribución de instrucciones ejecutadas y los ciclos de reloj que consume cada tipo:

Tipo de instrucción	**Ejemplos RV32I**	**Porcentaje del código**	**Ciclos (CPI_i)**
Aritmético-lógicas	add, addi, and, slli	45	1
Carga	lw, lbu	20	3
Almacenamiento	sw, sb	10	2
Saltos y bifurcaciones	beq, bne, jal, jalr	25	3

Se pide:

a. Calcular el CPI medio (ciclos por instrucción) del procesador para este programa.

$$\mathrm{CPI} = \sum_i \mathrm{C\,PI_i} \cdot \frac{\mathrm{I_i}}{\mathrm{I_c}} = \frac{(45\times1)+(20\times3)+(10\times2)+(25\times3)}{100} = 2$$

b. Calcular la velocidad de ejecución en MIPS.

$$\mathrm{MIPS} = \frac{\mathrm{f}}{\mathrm{CPI}\times10^6} = \frac{150\times10^6 Hz}{2\times10^6} = 75$$

c. Sabiendo que el programa tiene 450 millones de instrucciones, calcular el tiempo de CPU.

$$\mathrm{t_{CPU}} = \frac{\mathrm{I_c}\cdot\mathrm{CPI}}{\mathrm{f}} = \frac{(450\times10^6)\cdot2}{150\times10^6 Hz} = 6\ \mathrm{s}$$

Problema 2 (2,5 puntos)

Se desea mejorar el repertorio de instrucciones de un computador y se barajan varias alternativas, cada una de las cuales optimiza un único tipo de instrucción. La tabla recoge, para cada tipo, el porcentaje del tiempo total de ejecución que ocupa, el factor de mejora que se obtendría al aplicar la mejora (aislada) sobre ese tipo de instrucciones y su coste.

Tipo de instrucciones	Porcentaje del tiempo de ejecución	Factor de mejora	Coste en euros
Aritméticas	50	5	12.000
Carga y almacenamiento	30	2	3.000

Ejemplo de lectura de la primera opción: las instrucciones aritméticas ocupan el 50% del tiempo total del programa. Si se sustituye la ALU por otra más rápida, esas instrucciones se ejecutan 5 veces más rápido, y la sustitución cuesta 12.000 euros.

Suponiendo que el programa tarda 100 segundos en la ejecución original:

a. ¿Cuánto tardará el programa con la mejora que proporciona mayor ganancia?

$$T_A = 100\,(1 - 0{,}50) + 100\left(\frac{0{,}50}{5}\right) = 60\ s$$

$$T_{C\ y\ Al} = 100\,(1 - 0{,}70) + 100\left(\frac{0{,}30}{2}\right) = 85\ s$$

b. ¿Cuál es el coste por cada segundo ganado en cada opción?

A: 12.000/40 = 300 euros
B: 3.000/15 = 200 euros

c. Si sólo puede incorporarse una mejora, ¿cuál elegiría y con qué criterio?

Si hay restricciones económicas, B

Si la mejora técnica es ineludible, A

Problema 3 (1 punto)

Un procesador RISC-V (RV32I) segmentado completa de media 300 millones de instrucciones por segundo. Cada instrucción permanece en el procesador, desde que se busca hasta que escribe su resultado, una media de 40 ns. El sistema está en régimen estable, es decir, sale el mismo número de instrucciones por segundo que entra.

¿Cuántas instrucciones hay de media en vuelo dentro del procesador?

$L = \lambda \cdot W$= 300× $10^6\ \frac{1}{s} \times\ 40 \times 10^{-9} s = 12$

Ejercicio de traza (4 puntos)

Se muestra el siguiente programa RISC-V para el simulador RARS con su configuración por defecto. Con esa configuración, el código empieza en 0x00400000 y sp vale 0x7FFFEFFC. La memoria de datos está inicializada a 0.

Dirección	Etiqueta	Instrucción
0x00400000		j main
0x00400004	Triple:	addi sp, sp, -8
0x00400008		sw s1, 4(sp)
0x0040000c		sw s2, 0(sp)
0x00400010		slli s1, a0, 2
0x00400014		sub s2, s1, a0
0x00400018		mv a0, s2
0x0040001c		lw s1, 4(sp)
0x00400020		lw s2, 0(sp)
0x00400024		addi sp, sp, 8
0x00400028		jr ra
0x0040002c	main:	addi s1, zero, 5
0x00400030		addi s2, zero, 9
0x00400034		addi a0, zero, 4
0x00400038		jal ra, Triple
0x0040003c		add a0, a0, s1
0x00400040		sub a0, a0, s2
0x00400044		li a7, 1
0x00400048		ecall

Se pide:

a) Rellenar la tabla con los valores tras ejecutar cada instrucción indicada.
b) ¿Qué se imprime en la consola al terminar el programa?

Formato de respuesta:

- pc, ra, sp y las direcciones se escriben en hexadecimal de 8 dígitos (por ejemplo, 0x00400000).
- s1, s2, a0 y las palabras de memoria se escriben en decimal (por ejemplo, 25).

Registro / memoria	Tras jal ra, Triple	Tras sub s2, s1, a0	Tras jr ra
pc	0x00400004	0x00400018	0x0040003c

ra	0x0040003c	0x0040003c	0x0040003c
a0	4	4	12
s1	5	16	5
s2	9	12	9
sp	0x7fffeffc	0x7fffeff4	0x7fffeffc
Palabra en 0x7FFFEFF8	0	5	5
Palabra en 0x7FFFEFF4	0	9	9

Resultado final impreso en la consola: 8

INFORMACIÓN ÚTIL PARA RESOLVER EL EXAMEN

Ley de Amdahl:

$$T_{nuevo} = T\,(1 - f) + T\left(\frac{f}{N}\right)$$

$$Speedup\ (mejora)\ = \frac{1}{(1 - f) + \left(\frac{f}{N}\right)}$$

CPI medio:

$$CPI = \sum_{i} C\,PI_i \cdot \frac{I_i}{I_c}$$

MIPS:

$$MIPS = \frac{f}{CPI \times 10^6}$$

Tiempo de CPU:

$$t_{CPU} = \frac{I_c \cdot CPI}{f}$$

Ley de Little:

$$L = \lambda \cdot W$$

Instrucción	Sintaxis	Operación	Descripción
add	add rd, rs1, rs2	rd ← rs1 + rs2	Suma dos registros y guarda el resultado en rd.
addi	addi rd, rs1, imm	rd ← rs1 + imm	Suma un registro y una constante de 12 bits con signo (−2048 a 2047).
sub	sub rd, rs1, rs2	rd ← rs1 − rs2	Resta el registro rs2 del registro rs1 y guarda el resultado en rd.
slli	slli rd, rs1, shamt	rd ← rs1 << shamt	Desplazamiento lógico a la izquierda. Equivale a multiplicar por 2^shamt.
lw	lw rd, offset(rs1)	rd ← Mem[rs1 + offset]	Carga en rd la palabra de 32 bits de la dirección rs1 + offset.
sw	sw rs2, offset(rs1)	Mem[rs1 + offset] ← rs2	Almacena en memoria la palabra de 32 bits del registro rs2.
jal	jal rd, etiqueta	rd ← PC + 4; PC ← etiqueta	Salto incondicional con enlace. Guarda la dirección de retorno en rd (normalmente ra).
jalr	jalr rd, offset(rs1)	rd ← PC + 4; PC ← rs1 + offset	Salto indirecto con enlace. Se usa para retornar de funciones.
beq	beq rs1, rs2, etiqueta	si rs1 = rs2: PC ← etiqueta; si no: PC ← PC + 4	Salto condicional. Salta a la etiqueta si los dos registros son iguales; si no, ejecuta la siguiente instrucción.
ecall	ecall	Llamada al sistema	Pide un servicio al entorno. En RARS, a7 selecciona el servicio.

Pseudoinstrucción	Sintaxis	Se traduce a	Descripción
li	li rd, imm	addi rd, zero, imm	Carga una constante en un registro. Con constantes grandes se expande a lui más addi.
mv	mv rd, rs	addi rd, rs, 0	Copia el contenido de rs en rd.
j	j etiqueta	jal zero, etiqueta	Salto incondicional sin guardar dirección de retorno, porque zero descarta el valor.
jr	jr rs	jalr zero, 0(rs)	Salta a la dirección contenida en rs. jr ra es el retorno de función.

Servicio de ecall usado en RARS

a7	Servicio	Argumento
1	Imprimir entero	a0 = entero a imprimir

El servicio 1 de RARS (imprimir un entero con el valor en a0)